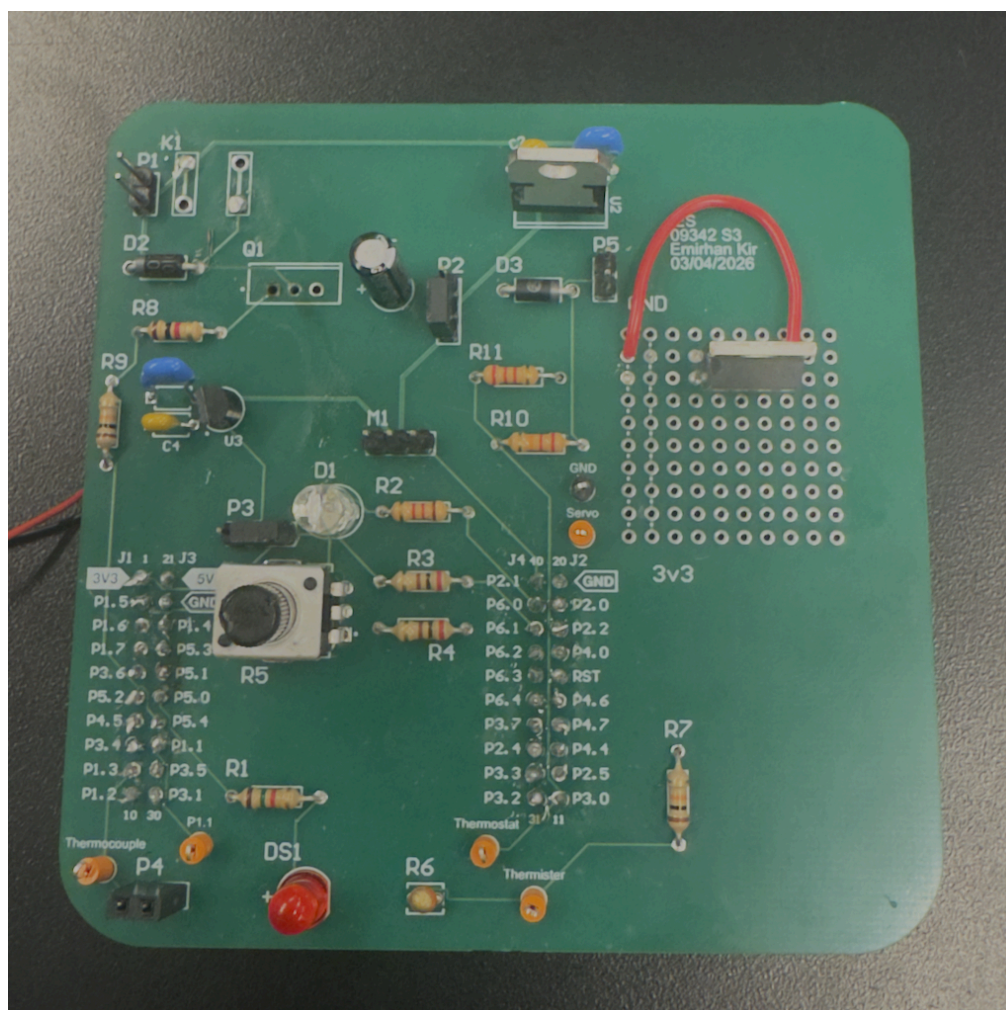




Gas Control Valve

Version Number

Date - 05/07/26

Section 3 - Group 1

Xavien Phillips

Emirhan Kir

James Arozamena

Table of Contents

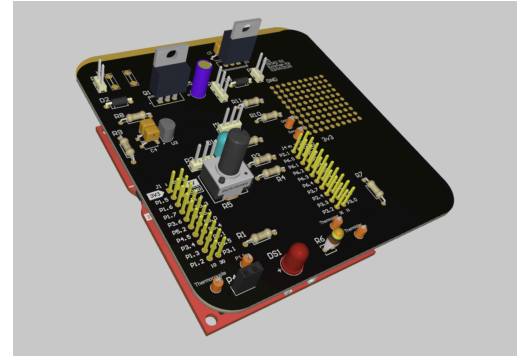
Table of Contents.....	2
Project Overview.....	4
Requirements, Features, and Constraints.....	5
Requirements.....	5
1. System Functionality.....	5
2. User Interface.....	5
3. Physical Needs.....	6
Features.....	6
Constraints.....	9
Tasks, Prioritization, and the Minimum Viable Product.....	11
Tasks.....	11
Physical.....	11
Software.....	11
Prioritization.....	11
Minimum Viable Product.....	13
Customer Requirements.....	13
Design Process and Tradeoffs.....	15
System Planning and Requirements.....	15
PCB Design Process.....	16
Component Placement Decisions.....	18
Software Design.....	19
Sensor and Output Testing.....	21
Debugging and Tradeoffs.....	22
Final Integration.....	24
Overall Design Reflection.....	25
Verification Processes.....	27
System Component Functionality.....	27
Design Issues and Debugging Approaches.....	32
UART.....	32
Thermistor.....	32
Potentiometer.....	32
Solenoid.....	33
Thermocouple.....	33
Design Impacts, Considerations, and Standards.....	34
Impacts and Considerations on/of Public Health, Safety, and Welfare.....	34
Impacts and Considerations on/of Global, Social, and Cultural Factors.....	34

Impacts and Considerations on/of the Operating Environment.....	35
Impacts and Considerations on/of Economic Factors.....	36
Applicable Standards.....	36
References.....	39

Project Overview

The objective of this project was to design and prototype a Gas Control Valve System using the MSP430FR2355 microcontroller.

The system automates temperature regulation while maintaining safe and efficient heating through flame monitoring and integrated safety shutdown mechanisms.



The controller operates using a six-state machine consisting of **IDLE, IGNITION, STABILIZATION, HEATING, SAFETY, and CLOSE_VALVE** states. The system waits for a thermostat call for heat before beginning ignition by opening the pilot solenoid and checking for flame detection using a thermocouple. After flame confirmation, the system enters the heating state, where temperature is regulated using sensor feedback and the user-defined setpoint. If unsafe conditions such as flame loss, failed ignition, or overheating occur, the system enters the **SAFETY** state and automatically closes the valves before returning to **IDLE**.

The prototype integrates a servo motor, solenoid, thermistor, thermocouple, potentiometer, thermostat input, RGB LED, and status LED. An ESP8266 Wi-Fi module was also added for remote monitoring and communication. A custom PCB was designed to integrate all required components while meeting the 99 mm × 99 mm size requirement.

Overall, the project successfully demonstrated automated ignition, valve control, flame monitoring, temperature regulation, and safety shutdown functionality through a reliable six-state control system.

Requirements, Features, and Constraints

Requirements

Our team began by consulting with the “customer” to understand the needs of the Gas Control Valve System. The first list of needs was broad, so we translated them into specific requirements that could be designed, tested, and verified later. The requirements were written as “the system shall” statements because each one needed to describe a clear action the system had to perform.

As the project developed, some requirements became more detailed, especially for flame detection, valve control, LED feedback, and safety shutdown. We organized the requirements into three main groups: **System Functionality**, **User Interface**, and **Physical Needs**.

1. System Functionality

- a. The system shall detect flame presence.
- b. The system shall control a pilot valve and a main valve.
- c. The system shall perform an ignition sequence when the call for heat is active.
- d. The system shall shut down automatically after three failed ignition attempts.
- e. The system shall monitor internal temperature.
- f. The system shall close the pilot valve and main valve if flame is lost.
- g. The system shall enter a safety state if the internal temperature exceeds the maximum threshold.

2. User Interface

- a. The system shall provide visual status feedback using an RGB LED and a single LED.

- b. The system shall allow the user to set the desired temperature with the potentiometer.
- c. The system shall allow remote monitoring and control through UART/Wi-Fi communication.
- d. The system shall show different system states through LED feedback.

3. Physical Needs

- a. The system shall be built around the MSP430FR2355 LaunchPad.
- b. The custom PCB shall fit within a 99 mm x 99 mm space.
- c. The system shall integrate the required components for ignition, temperature control, valve control, and visual feedback.
- d. The PCB shall allow the main inputs and outputs to be tested during development.

Features

Our prototype incorporates several hardware and software features that were required to meet the system requirements listed above. During the initial design phase, we identified a range of hardware components capable of supporting the Gas Control Valve System. These components were evaluated based on their compatibility with the system, and the final selections were made according to availability, performance, and cost.

This includes:

- **RGB LEDs:** Satisfying requirement 2.a, these are used to display the current operating state of the system and provide visual error indications when faults occur.
- **Red LED:** Satisfying requirement 2.a, this signals when there is a call for heat from the thermostat and also indicates system faults.

- **Servo:** Satisfying requirement 1.b, this acts as the main gas valve by opening and closing as required during operation.
- **Solenoid:** Satisfying requirement 1.b, this serves as the pilot valve that opens during ignition and remains closed during all other operating states.
- **Thermistor:** Satisfying requirement 1.e, this monitors the internal system temperature to help prevent overheating conditions.
- **Thermocouple:** Satisfying requirement 1.a, this is responsible for detecting the presence of a flame during ignition and heating operation.
- **Potentiometer:** Satisfying requirement 2.b, this allows the user to adjust the desired system temperature.
- **Wifi Module:** Satisfying requirement 2.c, this enables communication between the system and the customer's device for remote monitoring and control functions.

The software design utilizes timers to track failed ignition attempts in accordance with requirement 1.d. In addition, an Analog to Digital Converter (ADC) is used to process the potentiometer input for system temperature control, satisfying requirement 2.b.

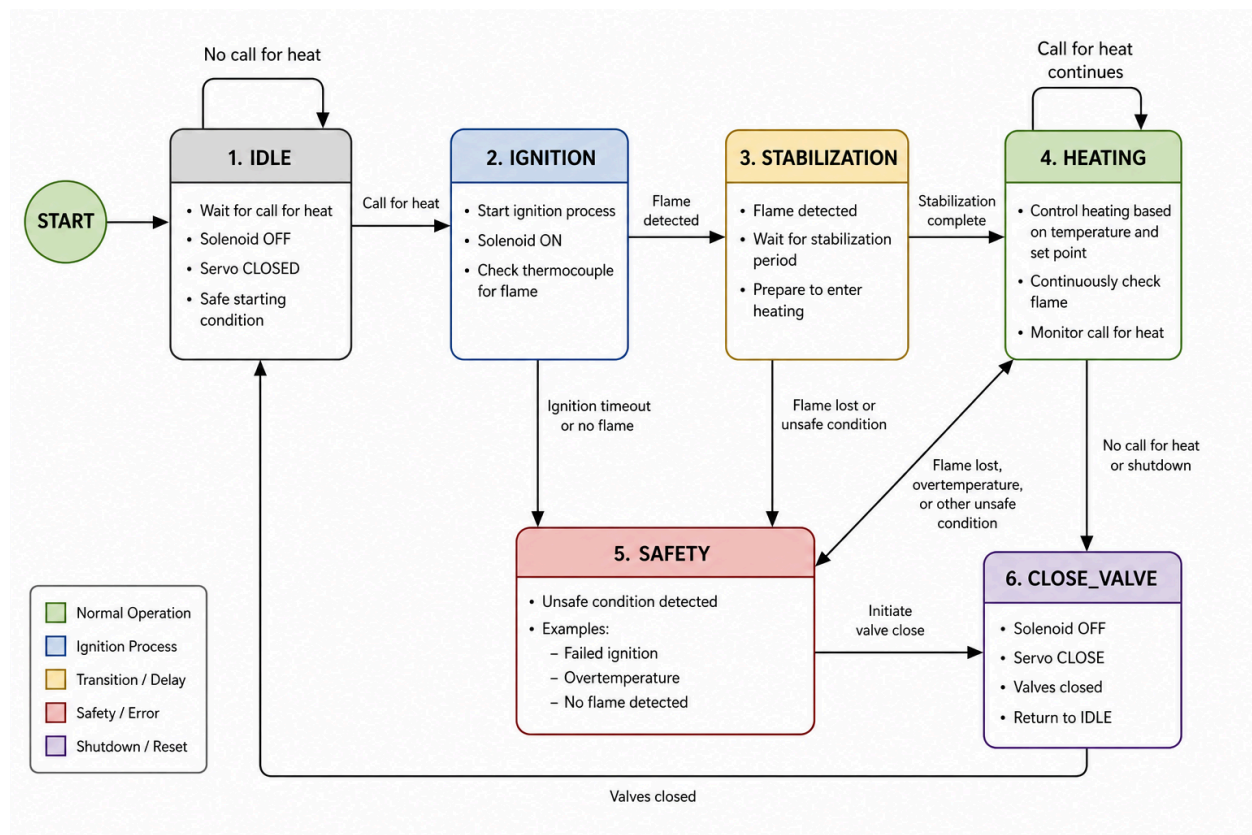


Figure 1: System States Diagram

We showed and explained how the system operates by developing a diagram that outlines each operating state along with its associated functions and transitions. The system consists of six states: four used for normal operation and two used for safety/shutdown behavior.

- **State 1: IDLE**

- The system waits for a call for heat. The solenoid is off, the servo is closed, and the system stays in a safe starting condition.

- **State 2: IGNITION**

- The system starts the ignition process. The solenoid turns on, and the thermocouple is checked for flame detection.

- **State 3: STABILIZATION**

- After flame is detected, the system waits for a short stabilization period before entering normal heating.
- **State 4: HEATING**
 - The system controls heating based on the temperature reading and set point. It also keeps checking for flame and calls for heat.
- **State 5: SAFETY**
 - The system enters this state if an unsafe condition happens, such as failed ignition, overtemperature, or no flame detection.
- **State 6: CLOSE_VALVE**
 - The solenoid turns off, and the servo closes. After the valves are closed, the system can return to IDLE.

Constraints

One major constraint was on the PCB we created. Once the board was ordered and soldered, routing mistakes were harder to fix. The biggest example was the thermistor trace using P1.6 and P1.7, which were later needed for UART. Since the trace was on the bottom layer, it could not be easily cut without risking damage to the board. Because of that, UART and Wi-Fi had to be tested through back channels instead of the normal board route.

Another constraint was the thermocouple signal. The thermocouple output was too small to read directly, so the SAC had to be used to amplify the signal before the ADC could read it. This made the design more complicated, but it was needed for flame detection to be reliable.

The solenoid was also a constraint because it is a higher-power switching output compared to the sensor inputs. We had to keep the solenoid control separated from sensitive analog readings as much as possible so it would not affect the sensor values.

Time also affected what we focused on. Instead of making every feature perfect, we focused on the minimum system that could show safe operation: read the sensors, control the valves, show LED feedback, and enter safety states. The final demo showed that the thermistor, solenoid, ignitor LED, RGB LED, servo, and UART/Wi-Fi worked, but the thermocouple and potentiometer were only partially complete in some areas.

Tasks, Prioritization, and the Minimum Viable Product

Tasks

Physical

- **Component Selection:** Correctly choosing each component of the PCB based on the most cost-effective available option.
- **Pin Mapping:** Connected each component with a pin on the MSP430 that best fits its technical requirements.
- **PCB Design:** Designed PCB based on the specific electrical needs of each component
- **Testing:** Placed certain components, like the thermistor and thermocouple, closer to the edge of the board for easy testing

Software

- **State Machine Implementation:** Developing a 6-state machine (IDLE, IGNITION, STABILIZATION, HEATING, SAFETY, CLOSE_VALVE) to manage system logic.
- **Peripheral Configuration:** Using ADC for analog sensors (thermistor, potentiometer), PWM for the actuators (servo, RGB LED), and GPIO for digital signals (solenoid, thermostat)
- **Communication:** Configure the UART to send status information for Wi-Fi monitoring

Prioritization

Foundational Planning

- The first priority was finishing the pin map, block diagram, and Altium schematic.
 - These were needed before the PCB could be routed and before the code could match the hardware.

PCB and Hardware First

- The PCB had to be designed and ordered early because it had the longest turnaround time.
 - Once the board arrived, soldering and hardware testing became the next priority.

Critical Outputs

- The servo, solenoid, and LEDs were tested early because they showed if the system could actually control the valve sequence.
 - The servo represented the main valve, the solenoid represented the pilot valve, and the LEDs showed the system state.

Critical Inputs

- The thermistor, thermocouple, potentiometer, and thermostat were tested after the main outputs.
 - These inputs were needed for temperature control, flame detection, set point control, and call for heat.

Safety Logic

- The state machine was the most important software task.

- The main valve could not open until flame was confirmed, so flame detection and valve control were treated as critical dependencies.

Lower Priority Features

- UART/Wi-Fi was tested later because communication was useful, but it was not more important than the safe valve control.
 - Since UART needed P1.6 and P1.7, the PCB routing issue made this harder and forced us to use back-channel testing.

Future Prioritization Strategy

- If we did the project again, we would prioritize the pin map and PCB routing earlier.
 - We would reserve UART pins from the start, group related parts better, and organize test points before ordering the board.

Minimum Viable Product

Customer Requirements

- **Core Ignition:** The system must open the pilot solenoid and activate the spark ignitor in response to the call for heat.
- **Flame Detection:** The system must detect and confirm the presence of a flame using the thermocouple before opening the main gas valve (servo).

- **Safety Shutdown:** The ability to stop gas flow if the flame is abruptly lost or if the ignition sequence fails three times.
- **User Interface:** The ability to visually show the customer what state the machine is in based on the color of the single LED and the RGB LED
- **Control & Monitoring:** The ability to set the temperature using the potentiometer as well as monitor the internal temp using the thermistor.

Design Process and Tradeoffs

System Planning and Requirements

Our team started by breaking the gas control valve system into the main functions that had to work.

The system needed to:

- Read a call for heat
- Check temperature
- Detect flame
- Control the pilot valve and main valve
- Give user feedback through LEDs
- Shut down if something unsafe happens

Since safety was the main focus, we designed the system so the valves stayed closed unless the correct conditions were met.

The main inputs were:

- **Thermostat:** call for heat
- **Thermistor:** temperature reading
- **Potentiometer:** set point
- **Thermocouple:** flame detection

The main outputs were:

- **Solenoid:** pilot valve
- **Servo:** main valve
- **Single LED:** heating/status feedback
- **RGB LED:** system state feedback
- **UART/Wi-Fi:** status communication

This helped us turn the requirements into parts we could actually design, code, and test.

PCB Design Process

The first major step was creating the components in Altium. With Karl Dyer and Michelle Froio, we built schematic blocks for:

- Thermistor
- Thermocouple
- Potentiometer
- Thermostat
- Solenoid
- Servo
- RGB LED
- Single LED
- Power circuit

After the schematic blocks were completed, we connected them into the top-level schematic and created the PCB layout. The goal was to make a custom board that could hold the needed hardware and make testing cleaner than loose wiring.

During PCB layout, the main design choices were:

- Keep the MSP430 header near the center of the board
- Route inputs and outputs to both sides of the board
- Place sensor inputs mostly near the lower/left side
- Keep output devices close to their MSP430 pins
- Keep the solenoid circuit grouped with its MOSFET and diode
- Add labels like 3V3, GND, Servo, Thermistor, and Thermocouple

The PCB was peer-reviewed in class before ordering. During the review, we checked:

- Routing
- Component placement
- Labels
- Pin conflicts
- Silk to solder warnings
- Other possible layout issues

After the review, we made updates and submitted the board for ordering. Once it arrived, we soldered the components and started testing each section.

Tradeoff:

The PCB worked for testing, but the layout could have been cleaner. Some related parts were spread out, and some traces were longer than needed. We also had an accidental trace issue between P1.6 and P1.7, which caused problems later when the UART needed those pins.

In a future revision, we would:

- Group related parts better
- Reserve communication pins earlier
- Make test points more organized
- Keep important traces easier to access
- Reduce long diagonal traces

Component Placement Decisions

Some important placement decisions were made to make testing and routing easier.

Thermistor and thermocouple

- Placed near the edge of the board
- Easier to access during testing
- Both needed to be checked while the system was running

RGB LED

- Placed near P6.0, P6.1, and P6.2
- Kept the RGB traces shorter
- Made each color channel easier to test

Solenoid circuit

- Placed near the MOSFET and diode
- Kept the switching circuit grouped together

- Made debugging easier because the solenoid depended on the diode, MOSFET, power path, and GPIO signal

Servo connector

- Placed near the middle of the board
- Routed to P2.1 for PWM control
- Kept the main valve output close to the MSP430 header

Tradeoff:

Not every part ended up in the best location. The board was still useful for testing, but a cleaner version would have better grouping between inputs, outputs, power, and test points.

Software Design

For the software, we split the code into separate driver files instead of putting everything into one large file.

Separate files were used for:

- Servo
- Solenoid
- Thermistor
- Thermocouple
- RGB LED
- Potentiometer
- Thermostat

- UART
- ADC

This made the code easier to test because each file had one main job.

A big part of the software design was deciding which outputs should use **GPIO** and which should use **PWM**. GPIO was used for simple on/off parts like the solenoid and single LED because they only needed to be either HIGH or LOW. PWM was used for the servo and the RGB LED because those parts needed more control. The servo needed PWM to control valve position, and the RGB LED needed PWM to control the brightness of each color channel.

The main peripheral decisions were:

- **ADC** was used to read analog inputs like the thermistor, thermocouple, and potentiometer.
- **GPIO** was used for simple digital inputs and outputs, like the thermostat input, solenoid, and single LED.
- **PWM** was used for outputs that needed timing or variable control, like the servo and RGB LED.
- **UART** was used to send status information for Wi-Fi communication.

The biggest software decision was using a **state machine**. This made the behavior code easier to follow because each state had a clear purpose.

The main states were:

- **IDLE**

- **IGNITION**
- **HEATING**
- **STABILIZATION**
- **SAFETY**
- **CLOSE_VALVE**

The basic state flow was:

- **IDLE:** system waits with valves closed
- **IGNITION:** solenoid opens, and flame is checked
- **STABILIZATION:** short delay after flame is detected
- **HEATING:** system controls valve based on temperature
- **SAFETY:** The system shuts down if something unsafe happens
- **CLOSE_VALVE:** valves close, and the system returns to idle

Tradeoff:

The state machine took more planning, but it made the system safer and easier to debug. It also made it easier to explain what the system was doing at each point.

Sensor and Output Testing

After the PCB was built, we tested each component separately before running the full behavior code. This helped us find problems one part at a time.

Servo and RGB LED

- Tested first because both used PWM

- The servo was tested by changing the pulse width
- The RGB LED was tested by checking red, green, and blue separately

Potentiometer

- Tested as the set point input
- Confirmed the value changed when the knob was turned

Thermistor

- Tested through the ADC
- Confirmed it could read temperature changes

Thermocouple

- Tested for flame detection
- Needed extra attention because the signal was small

Thermostat

- Tested as the call for heat input

Solenoid

- Tested by switching it on and off through the GPIO

This testing process helped us narrow down issues. When something did not work, we could test that part by itself instead of guessing where the issue was in the full system.

Debugging and Tradeoffs

The main issues were a mix of software, PCB routing, and hardware.

UART pin conflict

- UART needed P1.6 and P1.7
- Our PCB had a thermistor trace involving those pins
- The trace was on the bottom layer, so cutting it could have damaged the board
- We tested UART and Wi-Fi through back channels instead

Tradeoff:

This was not the cleanest solution, but it let us prove communication worked without risking the PCB.

Thermocouple signal

- The thermocouple output was small
- The SAC was needed to make the signal readable by the ADC
- This made the setup more complicated

Tradeoff:

It added extra setup, but flame detection was too important to ignore.

Solenoid debugging

- The solenoid was a higher-power switching output
- We had to check the code, soldering, diode orientation, power path, and switching circuit
- This showed that the issue was not always only in the code

RGB LED debugging

- The colors did not always come out correctly at first
- We checked the code, timer outputs, and compare registers
- Testing one color at a time made the issue easier to find

Final Integration

Once the individual parts worked, we combined everything into the behavioral code.

The final code connected:

- Thermostat input to start the call for heat
- Thermistor to check temperature
- Potentiometer to set the desired value
- Thermocouple to check flame
- Solenoid to control the pilot valve
- Servo to control the main valve
- LEDs to show system status
- UART/Wi-Fi to send status information

The first full behavior test was only somewhat successful because the thermocouple was not detecting flame correctly. Since the main valve should not open without flame detection, this had to be fixed before the system could be considered working.

After more testing, the behavioral code was successfully tested.

In the final demo, the system showed:

- Valve control
- Ignition detection
- Flame monitoring
- LED status feedback
- Safety state behavior
- Valve shutdown behavior
- UART/Wi-Fi communication

Overall Design Reflection

Our process went from requirements to schematic design, to PCB layout, to soldering, to individual component testing, and then to full system integration.

The biggest lesson was that embedded systems are not just code. The PCB, pin map, soldering, component orientation, sensor readings, and software all have to match.

The biggest tradeoff was balancing a working prototype with a perfect design. Our PCB worked for testing, but a future version would have:

- Cleaner routing
- Better grouping
- More organized test points
- Reserved UART pins
- Easier access to important traces

Even with those issues, the final system showed the main gas control valve sequence and met the main safety goals of the project.

Verification Processes

System Component Functionality

- **Thermister**
 - This component was tested using a multimeter. This is supposed to increase in voltage when heat is applied to measure the temperature of the board.

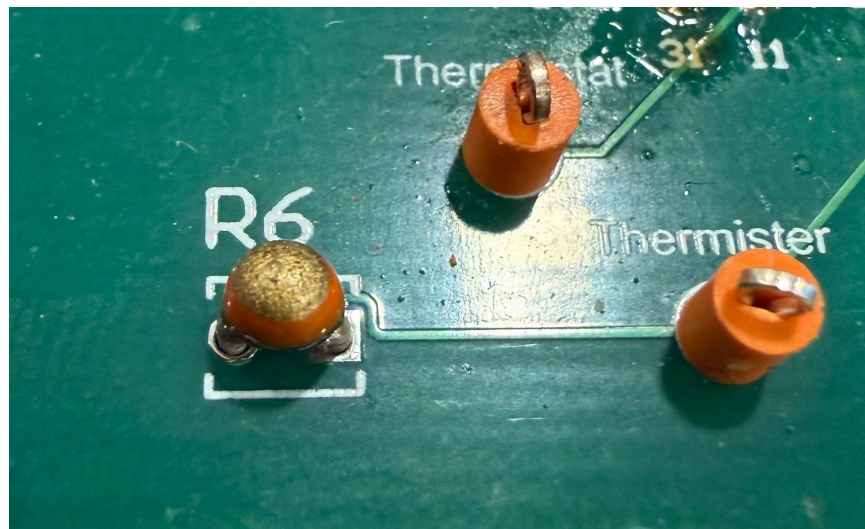


Figure 1. Thermister

- **Thermocouple**
 - This was also tested using the multimeter. The thermocouple is supposed to increase in voltage when heat is being applied to it. It was also tested by applying 3.3 volts to simulate a flame.

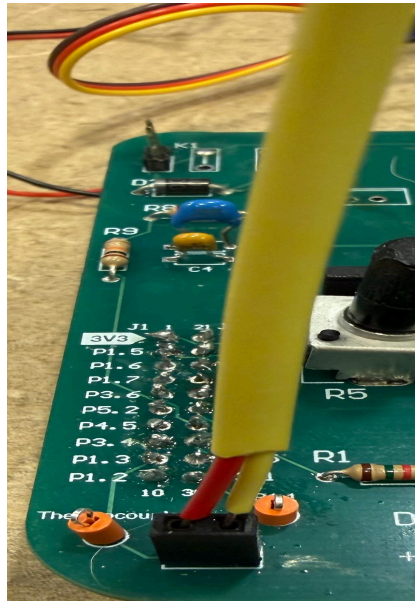


Figure 2. Thermocouple

- **Thermostat & Ignition LED**

- When voltage(3.3) is applied to the thermostat, it will light up the LED, stating that it is in the ignition state.

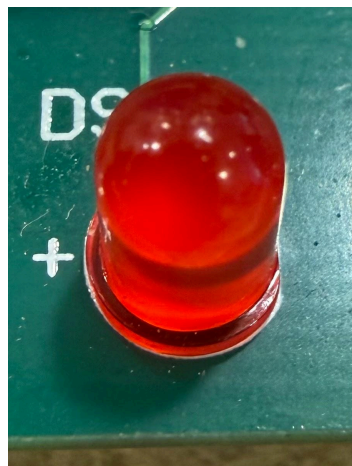


Figure 3. Ignition LED

- **Potentiometer**

- The potentiometer was tested first by adjusting its position to change the brightness of the RGB LEDs. Once that was completed, we moved on to setting the target temperature.

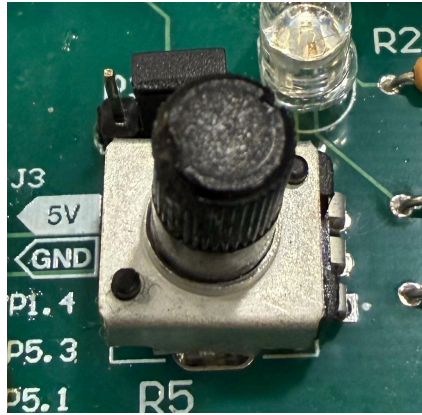


Figure 4. Potentiometer

- **Solenoid**

- The solenoid is able to open or close using the GPIO signal. This can be used to turn the pilot valve on or off based on the current state.

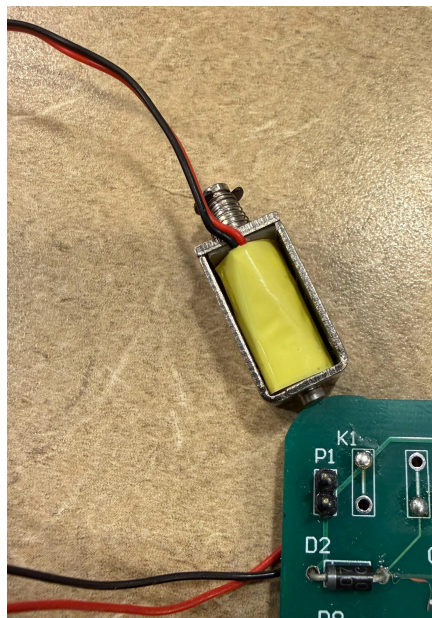


Figure 5. Solenoid

- **Servo**

- Correctly move 180 degrees back and forth to show the valve opening and closing to maintain heat.

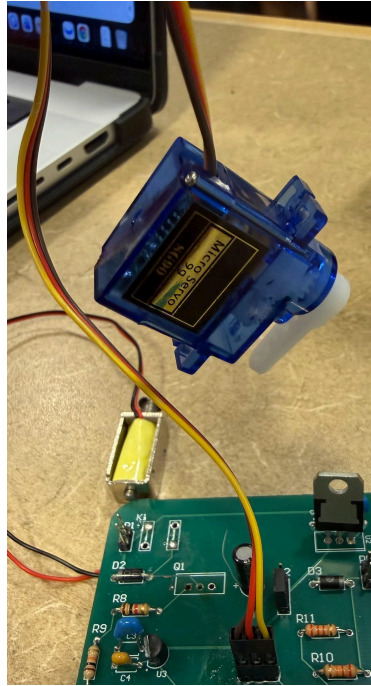


Figure 6. Servo

- **RGB LED**

- Correctly displays the colors red, blue, and green, and can be used to visually tell what the current state is.

- [RGB Testing](#)

- **UART**

- Display the current temperature of the board, whether the flame is detected, and what state that it is currently in to the serial monitor.

- **ADC**

- This correctly unlocks the digital converter, making peripherals switch from analog to digital.
- **Behavioral**
 - This correctly changes the states of systems based on temperature and whether a flame is detected or not, using switch states.

Design Issues and Debugging Approaches

UART

- When we first found out that the UART needed to be connected to pins 1.6 & 1.7 of the PCB board. This was a problem because those pins were being used for the thermistor. At first, we thought we could just cut the trace and retrace the thermistor with a different pin. After checking the trace, our group found out that it was a bottom-layer trace. We then discussed with Professor Trafford what we should do moving forward, and we decided to use the back channels to the serial monitor.

Thermistor

- When we were first making a code for the thermistor, we thought that it was just connected to pin 1.6. When we were trying to test this with the multimeter, it was not increasing in voltage based when heat was applied. After discussing with Professor Trafford, we found out that we had made an accidental secondary trace to pin 1.7. We then made pin 1.7 as an input to fix the issue.

Potentiometer

- The potentiometer would test successfully when controlling the brightness of the RGB LEDs. Once the group started coding the state machine, it would not allow us to use the potentiometer to set a temperature. Once we discussed with Professor Trafford, we realized that we did not set the correct channel in order to call on the potentiometer.

Solenoid

- The solenoid, when tested at first, was not opening and closing. After checking our code extensively, we discussed with Professor Trafford and found out that the wrong MOSFET was being used. Once we soldered on the correct MOSFET, the solenoid was correctly closing and opening.

Thermocouple

- The thermocouple would test successfully when using the multimeter. After integrating it into our behavioral code, it was not working properly. After looking back at our thermocouple code and the behavioral code, we found nothing wrong. We then found out after discussing with Professor Trafford and other groups that we were calling on the wrong channel, which did not allow us to read the thermocouple reading.

Design Impacts, Considerations, and Standards

Impacts and Considerations on/of Public Health, Safety, and Welfare

Since this project is based on a gas control valve system, safety was the biggest design consideration. Even though our prototype was not controlling real gas, the logic had to be treated as if it could. This is why we designed the system so that the valves stay closed by default and only open when the correct inputs are detected.

The main safety choices were:

- The servo and solenoid start in the closed/off position.
- The system checks for flame before opening the main valve.
- If flame is lost, the system closes the valve outputs.
- If the temperature gets too high, the system enters a safety state.
- LEDs give the user quick feedback about what state the system is in.

This matters because a real gas valve controller could create dangerous conditions if it opened a valve without ignition or failed to shut down after flame loss. Our design focused on making the system fail-safe instead of continuing to run during unsafe conditions. This connects directly to standards for gas systems, where safety shutoff and automatic ignition behavior are important parts of safe fuel gas operation.

Impacts and Considerations on/of Global, Social, and Cultural Factors

For the user side of the design, we used LED feedback because colors are easier to understand quickly than reading text or code values. This helps make the system easier to interpret for

different users. A person should be able to look at the board and tell if the system is idle, heating, or in a safe condition.

The RGB LED and single LED were used to make the system status more obvious. This is important because a warning or safety state should not depend on someone needing to understand the code or serial output. In a real product, the LED colors and blinking patterns would need to be consistent and clearly defined so users do not misread the system state.

This connects to IEC 60073, which gives coding principles for indicators and actuators so visual signals can be recognized more consistently and safely.

Impacts and Considerations on/of the Operating Environment

A real gas control valve system would likely be placed near heating equipment, such as in a basement, utility room, or mechanical area. These environments can have heat, dust, moisture, vibration, and limited access. Because of that, the PCB and wiring would need to be protected better than our class prototype.

For our design, the operating environment affected several choices:

- Sensor readings needed to be reliable because heat and noise can affect analog values.
- The solenoid was treated as a switching output that could create noise.
- The thermocouple signal was small, so it needed amplification before being read.
- The PCB would need an enclosure in a real system.
- Wires and connectors would need strain relief so they do not loosen during use.

The main issue is that the board should not be exposed in a real operating environment. A final version would need a protective enclosure, better connector protection, and more testing around heat and electrical noise.

Impacts and Considerations on/of Economic Factors

Cost also affected the design. We used the MSP430FR2355 because it already included many of the features we needed, such as ADC, GPIO, PWM timers, UART, and the Smart Analog Combo. This reduced the need for extra external circuits.

Some economic decisions were:

- Using the SAC instead of adding an external amplifier for the thermocouple.
- Using LEDs for feedback instead of a more expensive display.
- Using a custom PCB to make testing cleaner and reduce wiring issues.
- Testing UART/Wi-Fi through back channels instead of risking damage to the PCB.
- Focusing on the minimum working safety sequence before adding extra features.

The tradeoff is that cheaper or simpler choices can also add debugging time. For example, the custom PCB helped organize the prototype, but routing mistakes were harder to fix once the board was ordered. In a real product, spending more time on PCB review before manufacturing would save money later by reducing rework.

Applicable Standards

- **Residential Thermostat / HVAC Call-for-Heat Standard**
 - The system was designed around a thermostat call-for-heat signal.

- In a real residential heating system, this would commonly use a 24VAC thermostat interface.
- For our prototype, we simulated this with 3.3V so the MSP430 could safely read the signal.
- This allowed us to test when the heating sequence should begin without using real 24VAC.
- **NFPA 54 / ANSI Z223.1: National Fuel Gas Code**
 - This applies because our project is based on a gas valve control system.
 - NFPA 54 is a safety code for fuel gas piping systems, appliances, equipment, and related accessories.
 - Our design connects to this by keeping valves closed by default, checking flame before opening the main valve, and closing valve outputs if flame is lost.
 - This supports the main safety goal of preventing unsafe gas flow.
- **UL 60730 / IEC 60730: Automatic Electrical Controls**
 - This applies because our system acts like an automatic electrical controller.
 - The MSP430 reads inputs, makes decisions, and controls outputs like the solenoid, servo, and LEDs.
 - A real version would need to be tested for safe operation, timing behavior, and failure response.
 - This connects to our state machine because the system must respond correctly during ignition, heating, stabilization, and safety shutdown.
- **IEC 60073: Indicator and Actuator Coding**

- This applies because our system uses a single LED and an RGB LED for user feedback.
- IEC 60073 gives general rules for assigning meanings to visual, acoustic, and tactile indications so that equipment can be monitored and controlled more safely.
- Our design connects to this through the LED state feedback for idle, ignition, heating, stabilization, and safety/error states.
- A real version would need consistent colors and blinking patterns so users can quickly understand the system state.
- **PCB and Electrical Safety Considerations**
 - Sensitive analog signals should be kept away from higher-power switching outputs.
 - This applies to our PCB because the thermistor and thermocouple readings could be affected by electrical noise.
 - The solenoid circuit should be isolated as much as possible from analog sensor readings.

References

- [1] Google, Gemini, “I am writing the Design Impacts, Considerations, and Standards section for the embedded systems final report about the Gas Control Valve System using an MSP430FR2355, a custom PCB which I have given the schematic for as a PDF, thermistor, thermocouple, solenoid, servo, RGB LED, single LED, and UART/Wi-Fi. Help me identify realistic public safety, user feedback, operating environment, economic, and applicable standards considerations. Do not write the full section for me. Instead, give me engineering points I should consider and explain how each one connects back to my design decisions. Use the requirement PDF’s I have included” Accessed on 3 May 2026. Google Gemini, large language model-based AI chatbot. Available at <https://gemini.google.com/>
- [2] National Fire Protection Association. “Free Access to NFPA Codes and Standards.” *NFPA*, www.nfpa.org/for-professionals/codes-and-standards/list-of-codes-and-standards/free-access. Accessed 3 May 2026.
- [3] UL Solutions. “IEC/UL 60730-1 6th Edition Standard: What Changes?” *UL Solutions*, www.ul.com/resources/iecul-60730-1-6th-edition-standard-what-changes. Accessed 4 May 2026.
- [4] Renesas. “IEC/UL 60730 Functional Safety for Home Appliances.” *Renesas*, www.renesas.com/en/applications/consumer-electronics/appliances/iecul-60730-functional-safety. Accessed 4 May 2026.
- [5] Industrial Monitor Direct. “Light Tower Color Standards: Industrial Automation Reference.” *Industrial Monitor Direct*, industrialmonitordirect.com/blogs/knowledgebase/light-tower-color-standards-industrial-automation-reference. Accessed 4 May 2026.

[6] PowerStream. “24VAC Tutorial.” *PowerStream*, 1 Aug. 2025,
www.powerstream.com/24vac-tutorial.htm. Accessed 7 May 2026.